

Skillab

AI Agent Development

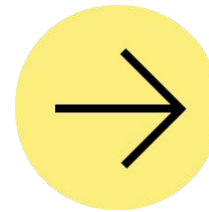
Richard Șerban Francu

Senior AI Engineer @UCE Systems



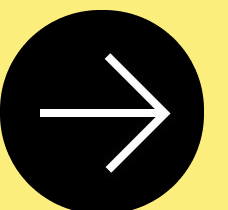
Programa cursului

1. Noțiuni de bază — API-uri LLM și primul agent
2. Instrumente, prompturi și observabilitate
3. Procesarea și extragerea documentelor
4. Stocarea bazelor de date și RAG
5. LangGraph — Stare și fluxuri de lucru
6. Orchestrarea multi-agent
7. Optimizarea ML — Dincolo de apelurile LLM
8. **Memorie și cache**
9. Securitate și măsuri de protecție
10. Integrarea MCP
11. Inferența SageMaker
12. Training, Fine-Tuning și RLHF



Memory și Caching

Cum facem agenții să "țină minte"
Optimizări pentru cost și latență
Durată: 2 ore



Problema: Agentul cu Alzheimer

Ce se întâmplă FĂRĂ memorie:

```
1  # ❌ FĂRĂ MEMORIE – fiecare apel e independent, LLM uită tot
2  reply = chat("Mă numesc Andrei.")
3  reply = chat("Cum mă numesc?")          # -> "Nu am această informație"
4
5
6  # ✅ CU MEMORIE – ținem istoricul într-o listă și-l retrimitem
7  history = []                             # ← memoria e o listă, atât
8
9  def turn(user_msg):
10     history.append({"role": "user", "content": user_msg})
11     reply = chat(history)                  # trimitem TOATĂ conversația
12     history.append({"role": "assistant", "content": reply})
13     return reply
14
15  turn("Mă numesc Andrei.")
16  turn("Cum mă numesc?")                  # -> "Te numești Andrei."
17
18  # Ideea: LLM-ul e stateless. Memoria o gestionăm NOI, în cod.
19
```



**Fiecare request =
conversație nouă
pentru LLM**

*Demo: agent fără
memorie vs cu
memorie*

De ce LLM-urile „uită”?

LLM-urile sunt **stateless by design** — fiecare API call e independent.

REQUEST 1

User: „Mă numesc Andrei”

LLM procesează → răspunde → **UITĂ**

Răspuns OK, dar fără persistență

REQUEST 2

User: „Care e numele meu?”

LLM: „*Nu am această informație*”

Fără context = fără memorie

- ✗ Nu există „sesiune” la nivel de LLM
- ✗ Contextul trebuie retrimis la fiecare request
- ✓ **NOI gestionăm memoria — în cod, nu în model**

Analogie: memoria umană

Cum translatăm conceptele biologice la agenți AI:

Memoria umană

Tip	Durată	Capacit.
Senzorială	< 1 sec	Mare
Short-term	20-30 s	~7 items
Long-term	Ani	Nelimitată

Echivalent în agentul AI

Memorie	Echivalent	Implementare
Short-term	Context window	Ultimele N msg
Working	Entity tracking	Dict cu entități
Long-term	DB / vector store	PostgreSQL, Pinecone



Cheia: fiecare tip de memorie umană are un mecanism tehnic distinct la agenți. *Nu există „one size fits all”.*

Context window: limita fundamentală

Fiecare LLM are o limită de tokeni pe care îi poate procesa simultan:

Model	Context	≈ pagini
GPT-3.5	4K / 16K	3-12
GPT-4	8K / 128K	6-100
Claude 3	200K	~150
Gemini 1.5	1M	~750



Dar context mare ≠ soluție



Cost

mai mulți tokeni = mai scump



Latență

mai mulți tokeni = mai lent



Lost in the middle

LLM ignoră informația din mijloc



Calitate

scade cu context prea mare

Soluția: memory management

În loc să trimitem TOT, trimitem CE E RELEVANT:

**FĂRĂ
MEMORY**



LLM

 *mulți tokeni, scump*

Trimitem toate cele 50 de mesaje la fiecare request →

**CU
MEMORY**

Memory System

- Sumarizare
- Selectare top-K
- Entity tracking



**Context
compact**



LLM

✓ *cost redus, latență mică*

Tipuri de Memory în LangChain

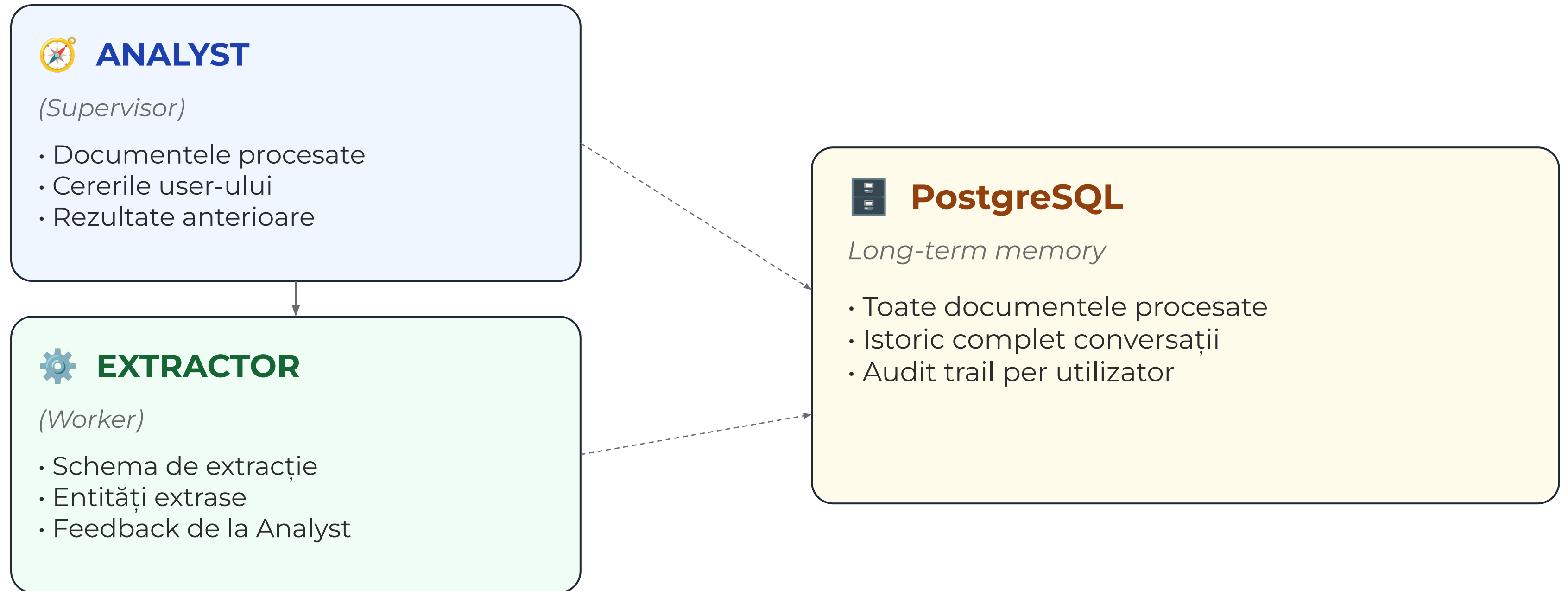
LangChain oferă mai multe strategii — fiecare cu trade-off-uri proprii:

Tip de memory	Cum funcționează	Când o folosești
ConversationBufferMemory	Păstrează TOATE mesajele	Conversații scurte
ConversationBufferWindowMemory	Doar ultimele N mesaje	Conversații lungi, context recent
ConversationSummaryMemory	Sumarizează periodic istoricul	Conversații foarte lungi
ConversationEntityMemory	Trackează entități (persoane, locuri)	Discuții despre subiecți recurenți
VectorStoreMemory	Caută semantic în istoric	Long-term, retrieval-based

 În lecția de azi explorăm primele 4. *VectorStoreMemory* = RAG, am acoperit-o în Lecția 4.

Memory în Document Analyst

Unde avem nevoie de memorie în proiectul nostru:



Memory vs Caching — care e diferența?

Două concepte diferite, ambele importante:

Aspect	Memory	Caching
Scop	Context pentru conversație	Evită recalcularea
Ce stochează	Mesaje, entități, fapte	Răspunsuri, embeddings
Când ajută	„Cine sunt eu?”	„Am mai răspuns la asta”
Reduce	Confuzie, repetiție	Cost, latență



MEMORY

„User-ul Andrei lucrează cu facturi de la Furnizor X”
→ agentul știe contextul



CACHING

„Întrebarea «ce e VAT?» a fost deja pusă” →
returnăm din cache, fără LLM

Roadmap Lecția 8

Ce vom acoperi împreună în următoarele 2 ore:

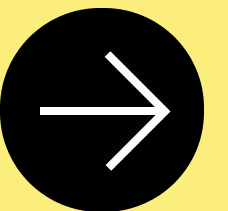
PARTEA 1 • MEMORY

- 1** **Intro** *(aici suntem)*
- 2** Conversation Memory
- 3** Entity Memory
- 4** Long-term (PostgreSQL)

PARTEA 2 • CACHING

- 5** Semantic Caching *(GPTCache)*
- 6** Prompt Caching *(Anthropic)*
- 7** Cache Strategies *& trade-offs*
- ★ Recap & Homework

Conversation Memory



Buffer și Window Memory

Cele mai simple două strategii de conversation memory:

```
1 store = {}
2 def get_history(session_id: str) -> InMemoryChatMessageHistory:
3     if session_id not in store:
4         store[session_id] = InMemoryChatMessageHistory()
5     return store[session_id]
6
7 chat = RunnableWithMessageHistory(
8     chain, get_history,
9     input_messages_key="input",
10    history_messages_key="history",
11 )
12
13 cfg = {"configurable": {"session_id": "andrei"}}
14 chat.invoke({"input": "Mă numesc Andrei."}, config=cfg)
```

```
1 store = {}
2 def get_history(session_id: str) -> WindowHistory:
3     if session_id not in store:
4         store[session_id] = WindowHistory()
5     return store[session_id]
6
7 chat = RunnableWithMessageHistory(
8     chain, get_history,
9     input_messages_key="input",
10    history_messages_key="history",
11 )
12
13 cfg = {"configurable": {"session_id": "andrei"}}
14 chat.invoke({"input": "Mă numesc Andrei."}, config=cfg)
15 print(chat.invoke({"input": "Cum mă numesc?"}, config=cfg).content)
16 # -> "Te numești Andrei." (tot istoricul e în store)
```

Tip	✓ Avantaj	✗ Dezavantaj
Buffer	Simplu, păstrează tot	Crește nelimitat → cost mare
Window	Size fix, cost predictibil	Pierde contextul vechi

Summary Memory

ConversationSummaryMemory — comprimă istoricul folosind un LLM:

```
1 class SummaryHistory(InMemoryChatMessageHistory):
2     """Când istoricul crește, comprimăm mesajele vechi într-un rezumat."""
3     def add_messages(self, messages: list[BaseMessage]) -> None:
4         super().add_messages(messages)
5         if len(self.messages) > MAX_MESSAGES:
6             old = self.messages[:-2]          # tot, mai puțin ultimul schimb
7             convo = "\n".join(f"{m.type}: {m.content}" for m in old)
8             summary = summarizer.invoke(
9                 f"Rezumă concis conversația, păstrând faptele cheie:\n{convo}"
10            ).content
11             # înlocuim mesajele vechi cu un singur SystemMessage-rezumat
12             self.messages = [SystemMessage(f"Rezumat anterior: {summary}")] + self.messages[-2:]
13
```

Mesaje originale (~150 tokeni)

„Andrei/TechCorp · 5 facturi PDF ·
extrag furnizor, sumă, dată...”

LLM sumarizează →

Summary (~25 tokeni — **6× mai compact**)

„Andrei/TechCorp procesează 5 facturi PDF, extrage:
furnizor, sumă, dată.”

✨ **ConversationSummaryBufferMemory** = *best of both worlds*

Summary pentru mesajele vechi

+ **Buffer** pentru mesajele recente

→ context dens + detalii recente păstrate intact

Care memory să aleg?

Trade-off-uri principale: tokeni vs. detalii păstrate vs. complexitate:

Memory Type	Tokeni	Detalii	Best for
Buffer	↗ crește	100%	Conversații scurte (<10 msg)
Window	— fix	Recent only	Chat support, task-uri rapide
Summary	↓ mic	Esență	Conversații foarte lungi
SummaryBuffer	≈ mediu	Mix	Document Analyst ✓

🎯 Reguli practice de selecție

Conversație scurtă? → Buffer

Conversație foarte lungă? → Summary

Context recent suficient? → Window (k=5)

Proiect complex (agenți)? → SummaryBuffer ✓

Implementare practică

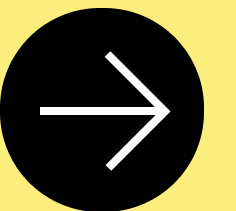
Două abordări — LangChain (LCEL) și LangGraph:

```
1
2 # 1. prompt | llm → pipeline-ul LCEL
3 prompt = ChatPromptTemplate.from_messages([
4     ("system", "Ești un asistent pentru analiză de documente."),
5     MessagesPlaceholder("history"),
6     ("human", "{input}"),
7 ])
8 chain = prompt | llm
9
10 # 2. RunnableWithMessageHistory înlocuiește vechiul
11 #   load_memory_variables() + save_context() – le face automat.
12 store = {}
13 chat = RunnableWithMessageHistory(
14     chain,
15     lambda sid: store.setdefault(sid, InMemoryChatMessageHistory()),
16     input_messages_key="input",
17     history_messages_key="history",
18 )
19
```

```
1 def agent(state: MessagesState) -> dict:
2     response = llm.invoke(state["messages"])
3     return {"messages": [response]} # ← reducer-ul îl adaugă la istoric
4
5 builder = StateGraph(MessagesState)
6 builder.add_node("agent", agent)
7 builder.add_edge(START, "agent")
8 builder.add_edge("agent", END)
9
10 # Checkpointer = persistența. Thread-ul (session) e identificat prin thread_id.
11 checkpointer = InMemorySaver()
12 graph = builder.compile(checkpointer=checkpointer)
13
```

Next → Secțiunea 3 · Entity Memory

Entity Memory



Ce este Entity Memory?

Problema cu Conversation Memory

Știe CE s-a spus, dar nu organizează DESPRE CINE / CE.

CONVERSATION MEMORY

Text flat, fără structură:

„Andrei lucrează la TechCorp și are 5 facturi de la Furnizor ABC...”

 Răspuns la „Cât am la ABC?” → *trebuie scanat tot textul.*

ENTITY MEMORY

Entități structurate cu attribute:



Andrei

- └ TechCorp · employee
- └ Task: facturi



TechCorp

- └ Angajat: Andrei



Furnizor ABC

- └ Tip: furnizor
- └ Documente: 5 facturi

Cum funcționează

ConversationEntityMemory rulează un mini-pipeline de 4 pași la fiecare interacțiune:

1

USER INPUT

„Andrei de la TechCorp are facturi de la ABC”

2

EXTRACȚIE (LLM)

Detectează: Andrei · TechCorp · ABC

3

UPDATE STORE

Adaugă / actualizează info pentru fiecare entitate

4

INJECT ÎN PROMPT

„Țtii despre: Andrei (angajat TechCorp)...”



Trade-off: +1 LLM call per interacțiune (pasul 2 — extracția entităților)

Implementare

ConversationEntityMemory — folosește LLM-ul intern pentru extracție:

```
1
2 class Entity(BaseModel):
3     name: str = Field(description="numele entității")
4     type: str = Field(description="persoană / companie / furnizor")
5     facts: list[str] = Field(description="fapte despre entitate")
6
7 class Entities(BaseModel):
8     entities: list[Entity]
9
10 extractor = llm.with_structured_output(Entities)
11
12 entity_store: dict[str, Entity] = {}
13 def remember(text: str):
14     for e in extractor.invoke(f"Extrage entitățile din: {text}").entities:
15         entity_store[e.name] = e      # update/insert pe nume
16
17 remember("Andrei de la TechCorp are 5 facturi de la Furnizor ABC")
18 print(entity_store["ABC"])
```



save_context()

Trimite mesajul către LLM, care extrage entitățile.



entity_store.store

Dict cu entități și descrieri textuale, accesibil direct.



load_memory_variables()

Returnează DOAR entitățile relevante pentru query-ul curent.

Entity Memory vs Conversation Memory

Două abordări complementare — fiecare cu cazurile ei de utilizare:

Aspect	Conversation Memory	Entity Memory
Stochează	Mesaje raw	Entități structurate
Organizare	Cronologică	Pe entitate
Query tipic	„Ce s-a spus?”	„Ce știm despre X?”
LLM calls extra	0	1 per interacțiune
Best for	Dialog general	CRM, knowledge bases

✓ **Folosește Entity Memory când...**

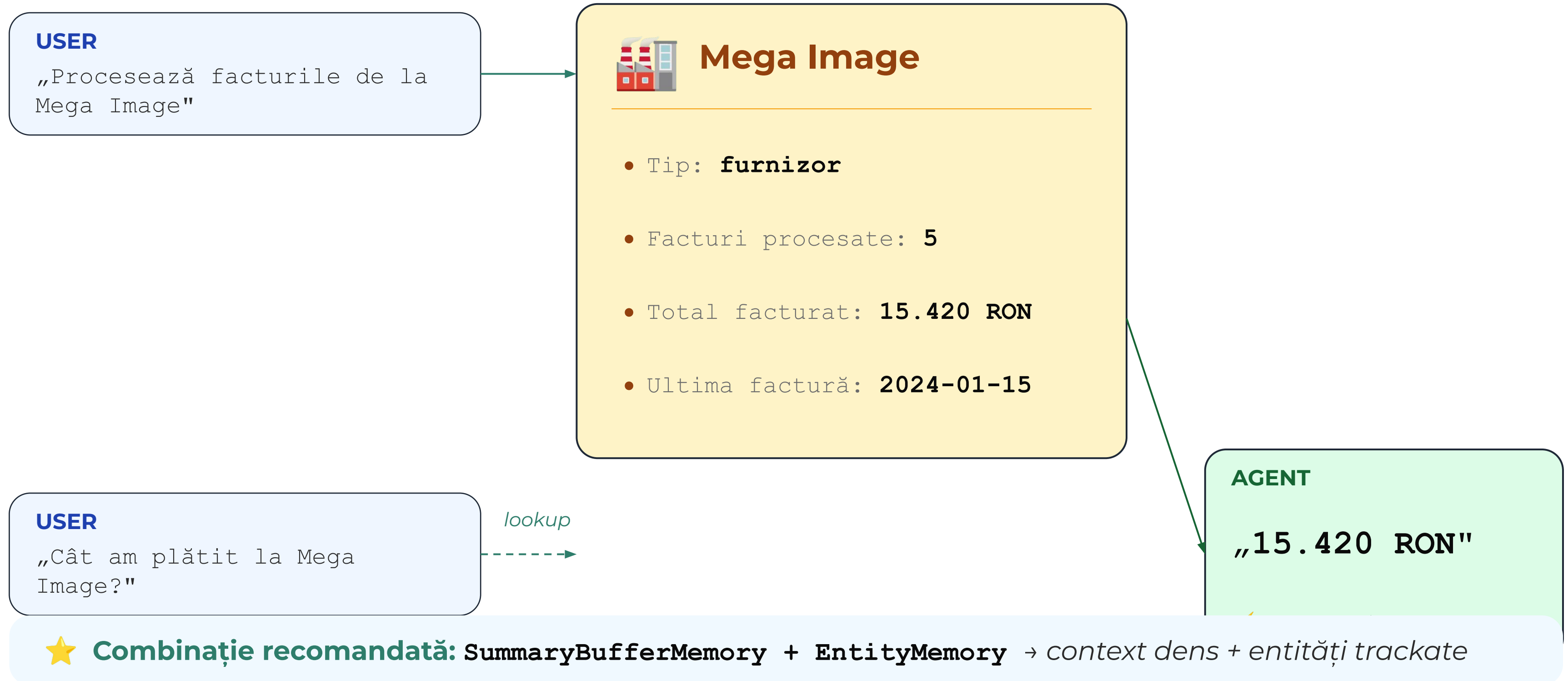
- Lucrezi cu persoane / companii recurente
- Document Analyst (furnizori, clienți)
- Răspunzi des la „Ce știm despre X?”

✗ **Evit-o când...**

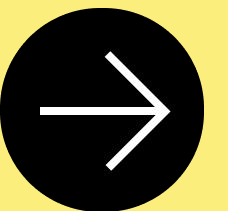
- Conversațiile sunt scurte și simple
- Nu există entități persistente în domeniu
- Costul extra LLM calls nu se justifică

Entity Memory în Document Analyst

Aplicare practică — track furnizori și răspunsuri instant din entity store:



Long-term Storage (PostgreSQL)



Problema: memoria se pierde

Ce se întâmplă când restartezi aplicația — fără persistență, totul dispare:

SESIUNEA 1

✓ *memorie activă*

User: „Mă numesc Andrei, procesez facturi TechCorp”

Agent: „OK, am reținut!”

Memory: {Andrei: „angajat TechCorp”, ...}



RESTART APP · *memoria dispare*

SESIUNEA 2

✗ *memorie pierdută*

User: „Continuă cu facturile”

Agent: „Ce facturi? Cine ești?”

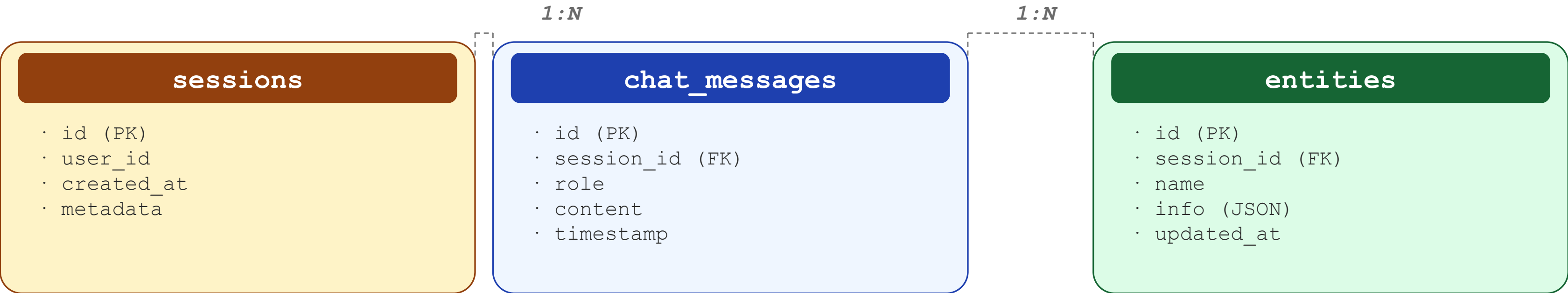
Memory: {} ← **PIERDUTĂ**

Ce persistăm?

4 tipuri de date pe care le salvăm în PostgreSQL pentru continuitate între sesiuni:

Tip	Ce stocăm	Tabel
Conversații	Mesaje user / assistant	chat_messages
Entități	Info despre persoane / companii	entities
Sesiuni	Metadata sesiune	sessions
State LangGraph	Checkpoints workflow	graph_checkpoints

Schema de bază — 3 tabele relaționate:



Implementare: schema SQLAlchemy

Definim 3 modele ORM care mapează tabelele PostgreSQL:

```
1 class ChatMessageRepository:
2     def __init__(self, db: DBSession):
3         self.db = db
4
5     def add(self, session_id: str, role: str, content: str) -> ChatMessage:
6         msg = ChatMessage(session_id=session_id, role=role, content=content)
7         self.db.add(msg)
8         return msg
9
10    def latest(self, session_id: str, limit: int) -> list[ChatMessage]:
11        # ORDER BY timestamp DESC + LIMIT N → ultimele N (window)
12        stmt = (
13            select(ChatMessage)
14            .where(ChatMessage.session_id == session_id)
15            # id ca tiebreaker: mesajele scrise în aceeași secundă au timestamp
16            # identic, iar id-ul auto-increment garantează ordinea reală.
17            .order_by(ChatMessage.timestamp.desc(), ChatMessage.id.desc())
18            .limit(limit)
19        )
20        rows = self.db.execute(stmt).scalars().all()
21        return list(reversed(rows))      # reverse → cronologic pentru LLM
22
```



Session

Un id unic per conversație. Ține user_id și metadata.



ChatMessage

role + content + timestamp. Foreign key către Session.



Entity

name + info (JSON). Index pe name pentru lookup rapid.

Memory Manager: Load & Save

PersistentMemory — un wrapper care încarcă din DB și salvează după fiecare interacțiune:



```
1 # — Manager: API curat pentru restul aplicației. Nu știe de SQL.
2 class PersistentMemory:
3     def __init__(self, session_factory: sessionmaker, window: int = 10):
4         self.session_factory = session_factory
5         self.window = window
6
7     def load_messages(self, session_id: str) -> list[dict]:
8         with unit_of_work(self.session_factory) as db:
9             rows = ChatMessageRepository(db).latest(session_id, self.window)
10            return [{"role": m.role, "content": m.content} for m in rows]
11
12    def save_message(self, session_id: str, role: str, content: str) -> None:
13        with unit_of_work(self.session_factory) as db:      # tranzacție atomică
14            ChatMessageRepository(db).add(session_id, role, content)
15
```



load_messages()

- Query DB · ORDER BY timestamp DESC
- LIMIT N (window size)
- Map la HumanMessage / AIMessage
- Reverse → cronologic pentru LLM



save_message()

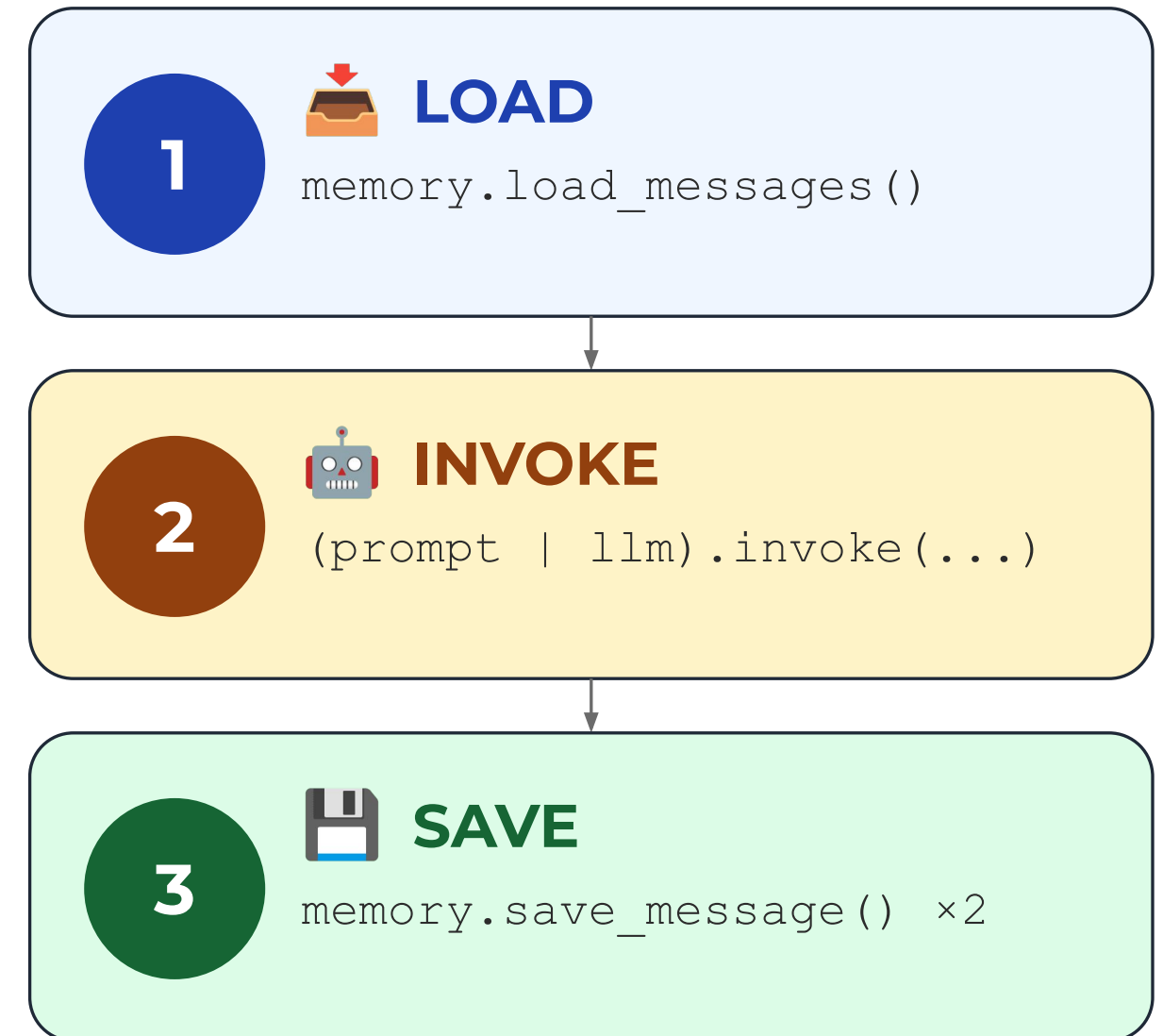
- Construiește ChatMessage(role, content)
- db.add() + db.commit()
- timestamp setat automat (default)
- *Apelat după FIECARE turn*



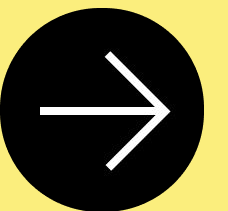
Integrare în Document Analyst

Pattern complet: *load* → *invoke* → *save* · memoria supraviețuiește restart-urilor:

```
1 class DocumentAnalyst:
2     def __init__(self, memory: PersistentMemory):
3         self.memory = memory
4         self.llm = ChatAnthropic(model="claude-sonnet-4-20250514", temperature=0)
5         self.prompt = ChatPromptTemplate.from_messages([
6             ("system", "Ești un asistent pentru analiză de documente."),
7             MessagesPlaceholder("history"),
8             ("human", "{input}"),
9         ])
10
11     def chat(self, session_id: str, user_input: str) -> str:
12         # 1. LOAD - istoricul din DB (ultimele N mesaje)
13         history = self.memory.load_messages(session_id)
14
15         # 2. INVOKE - prompt | llm cu istoricul injectat
16         chain = self.prompt | self.llm
17         reply = chain.invoke({"history": history, "input": user_input}).content
18
19         # 3. SAVE - persistăm ambele mesaje (x2), fiecare în tranzația lui
20         self.memory.save_message(session_id, "user", user_input)
21         self.memory.save_message(session_id, "assistant", reply)
22
23         return reply
24
```



Semantic Caching cu pgvector



Ce este Semantic Caching?

Cache tradițional verifică text identic; semantic cache verifică similaritatea de înțeles:

CACHE TRADIȚIONAL

(exact match)

„Ce este Python?”

HIT

„Ce e Python?”

MISS

SEMANTIC CACHE

(similarity match)

„Ce este Python?”

100%

HIT

„Ce e Python?”

~95%

HIT

„Explică-mi Python”

~90%

HIT

Cum funcționează:

Query: „Explică-mi Python”

1. Embedding

2. pgvector sim > 0.92?

DA

NU

✓ Return cache • \$0 • 10ms

✗ Call LLM • \$0.01 • 2s

Schema PostgreSQL pentru cache

Folosim tabel dedicat cu coloană Vector(384) pentru embedding-uri:

```
1 class SemanticCache(Base):
2     __tablename__ = "semantic_cache"
3
4     id: Mapped[int] = mapped_column(primary_key=True)
5     query_text: Mapped[str] = mapped_column(Text) # query original (debugging)
6     query_embedding: Mapped[list[float]] = mapped_column(Vector(384)) # vectorul
7     response: Mapped[str] = mapped_column(Text) # răspunsul cache-uit
8     created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), default=func.now())
9     hit_count: Mapped[int] = mapped_column(Integer, default=0) # de câte ori a fost servit
10
11     # Index HNSW pentru cosine distance – operatorul <=>.
12     # vector_cosine_ops TREBUIE să se potrivească cu distanța din query,
13     # altfel Postgres ignoră indexul și face full scan.
14     __table_args__ = (
15         Index(
16             "ix_semantic_cache_embedding_hnsw",
17             "query_embedding",
18             postgresql_using="hnsw",
19             postgresql_with={"m": 16, "ef_construction": 64},
20             postgresql_ops={"query_embedding": "vector_cosine_ops"},
21         ),
22     )
23
```

semantic_cache	
• id	PK
• query_text	string
• query_embedding	Vector(384)
• response	string
• created_at	datetime
• hit_count	int
+ HNSW index (vector_cosine_ops)	

De ce pgvector?



Reutilizare

Deja îl aveți din Lecția 4 (RAG)



Zero deps noi

Nu adăugați dependențe externe



Control total

Threshold + logică custom

Implementare Semantic Cache

SemanticCacheManager — clasă cu două metode: *get* (cu similarity search) și *set*:

```
1 class SemanticCacheManager:
2     def __init__(self, session_factory: sessionmaker, threshold: float = 0.92):
3         self.session_factory = session_factory
4         self.threshold = threshold      # similaritate minimă pt. HIT
5
6     def get(self, query: str) -> str | None:
7         """Caută cel mai apropiat query cache-uit. Returnează răspunsul sau None."""
8         emb = embed(query)[0]
9         with self.session_factory() as db:
10             # cosine_distance = 1 - cosine_similarity, deci ORDER BY ASC = cel mai similar.
11             distance = SemanticCache.query_embedding.cosine_distance(emb)
12             row = db.execute(
13                 select(SemanticCache, distance.label("dist"))
14                 .order_by(distance)      # folosește indexul HNSW
15                 .limit(1)
16             ).first()
17
18             if row is None:
19                 return None
20
21             cache, dist = row
22             similarity = 1 - dist
23             if similarity < self.threshold:
24                 return None              # cel mai bun candidat e prea diferit → MISS
25
26             cache.hit_count += 1         # contorizăm hit-ul
27             db.commit()
28             return cache.response
29
30     def set(self, query: str, response: str) -> None:
31         """Salvează un query + răspuns nou în cache."""
32         emb = embed(query)[0]
33         with self.session_factory() as db:
34             db.add(SemanticCache(query_text=query, query_embedding=emb, response=response))
35             db.commit()
36
```



get(query)

1. `embed_query(query)`
2. SQL `<=>` (cosine distance)
3. `filter > threshold`
4. `return top-1 sau None`



set(query, response)

1. `embed_query(query)`
2. `SemanticCache(query, emb, response)`
3. `db.add() + db.commit()`

Integrare în Agent

Pattern: check cache → call LLM (dacă miss) → save în cache:

```
1 class CachedDocumentAnalyst:
2     def __init__(self, cache: SemanticCacheManager):
3         self.cache = cache
4         self.llm = ChatAnthropic(model="claude-sonnet-4-20250514", temperature=0)
5
6     def query(self, user_input: str) -> str:
7         # 1. CHECK CACHE – întrebare similară deja răspunsă?
8         cached = self.cache.get(user_input)
9         if cached is not None:
10             return cached # HIT · $0 · ~10ms
11
12         # 2. CALL LLM – doar la MISS
13         response = self.llm.invoke(user_input).content # MISS · $$ · ~2s
14
15         # 3. SAVE CACHE – pentru viitoare query-uri similare
16         self.cache.set(user_input, response)
17         return response
18
```

1  **CHECK CACHE**
cache.get(input)

2  **CALL LLM**
doar la cache MISS

3  **SAVE CACHE**
cache.set(input, response)

Rezultate tipice:

 „Ce este VAT?”
LLM call · prima dată

 „Ce înseamnă VAT?”
Cache HIT · sim 94%

 „Explică-mi TVA”
Cache HIT · sim 91%

 „Capitala Franței?”
LLM call · topic diferit

Tuning și Best Practices

Threshold-ul controlează cât de „permisiv” este cache-ul; alege-l în funcție de domeniu:

Threshold	Comportament	Use case
0.98+	Foarte strict · aproape exact match	Date sensibile, precizie critică
0.92 – 0.95	Balans bun · recomandat	General purpose, FAQ ✓
0.85 – 0.90	Relaxat · mai multe hit-uri	Conversații casual

✓ Do's

- Cache răspunsuri deterministe (FAQ, explicații)
- Setează TTL pentru date care se schimbă
- Monitorizează hit rate (țintă: > 30%)
- Exclude query-uri cu date specifice

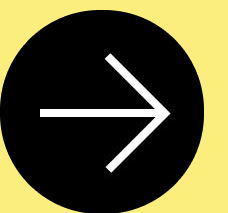
✗ Don'ts

- Cache răspunsuri personalizate per user
- Cache query-uri cu timestamp / date variabile
- Threshold prea mic → răspunsuri greșite
- Cache pentru analiză docs unice

⚠ Când NU folosești semantic cache:

query-uri unice · răspunsuri context-dependent · date real-time

Prompt Caching và Embedding Caching



Prompt Caching (provider-side)

Provider-ul cachează porțiunea statică a prompt-ului — următoarele request-uri plătesc doar fragmentul nou:

FĂRĂ PROMPT CACHE

R1:

Sys 2K

Doc 5K

Q

→ 7K t.

R2:

Sys 2K

Doc 5K

Q

→ 7K t.

Plătești integral de fiecare dată

CU PROMPT CACHE

R1:

Sys 2K

Doc 5K

Q

→ 7K t.

R2:

CACHED (10% cost)

Q

→ ~500 t.

→ până la 90% reducere pe input

Disponibilitate la cei 3 provideri majori (2026):

Provider	Discount	Activare	Min. tokeni
OpenAI	-50% input	Automat (zero cod)	1.024
Anthropic	-90% input	cache_control: ephemeral	1.024
Google Gemini	-75% input	API explicit + storage cost	32.768

Embedding Caching

Embedding-urile aceluiasi text sunt deterministe — calculează-le o singură dată, refolosește mereu:

✗ FĂRĂ CACHE → 100-200ms per call, load pe Ollama
recalculează același vector la fiecare apel

✓ CU CACHE → ~5ms lookup, calculat o singură dată
hash MD5 → SELECT din DB → return embedding

```
1 class EmbeddingCacheEntry(Base):
2     __tablename__ = "embedding_cache"
3     # PK = hash-ul textului → lookup O(1), text identic = aceeași cheie
4     text_hash: Mapped[str] = mapped_column(String(32), primary_key=True)
5     embedding: Mapped[list[float]] = mapped_column(Vector(384))
6
7
8 class EmbeddingCache:
9     def __init__(self, session_factory: sessionmaker):
10         self.session_factory = session_factory
11
12     @staticmethod
13     def _hash(text: str) -> str:
14         return hashlib.md5(text.encode("utf-8")).hexdigest()
15
16     def get_or_create(self, text: str) -> list[float]:
17         key = self._hash(text) # 1. MD5 hash
18         with self.session_factory() as db:
19             entry = db.get(EmbeddingCacheEntry, key) # 2. check DB
20             if entry is not None:
21                 return entry.embedding # HIT → ~5ms lookup
22
23             vector = embed(text)[0] # 3. MISS → generate
24             db.add(EmbeddingCacheEntry(text_hash=key, embedding=vector))
25             db.commit()
26             return vector
```

1 MD5 hash text

2 Check DB cache

3 Return / Generate



Sumar: ce cache folosim?

3 tipuri de cache complementare — fiecare adresează o problemă diferită:

Tip cache	Ce cachează	Provideri	Reducere
Semantic Cache	Răspuns LLM final	Orice (pgvector)	100% query similar
Embedding Cache	Vectori pentru text	Orice (PostgreSQL)	100% text identic
Prompt Cache	Input procesat de LLM	OpenAI · Anthropic · Gemini	50–90% input tokens

Strategia pentru Document Analyst — 3 layer-e complementare:

1

EMBEDDING CACHE · Chunk-urile documentelor cached → RAG retrieval mai rapid

toți providerii

2

SEMANTIC CACHE · Întrebări frecvente → „Ce e VAT?” returnat instant din cache

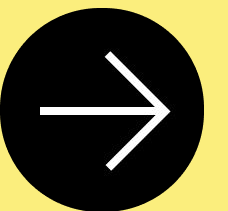
toți providerii

3

PROMPT CACHE · System prompt + context document cached la nivel de provider

OpenAI · Anthropic · Gemini

Cache Strategies & Trade-offs



Când să cachezi vs să recalculezi

Nu totul merită cached — analizează-ți cazurile cu „Regula celor 3R”:

✓ Cachează	✗ NU cacheza
Întrebări frecvente (FAQ)	Query-uri unice
Documente statice	Date real-time
System prompts	Răspunsuri personalizate
Embeddings pentru texte fixe	Texte care se schimbă

🎯 Regula celor 3R:

R Repetitiv?

Se repetă același query / document?

DA → **cachează** · NU → **nu merită**

R Relativ stabil?

Datele se schimbă rar?

DA → **cachează** · NU → **risc stale**

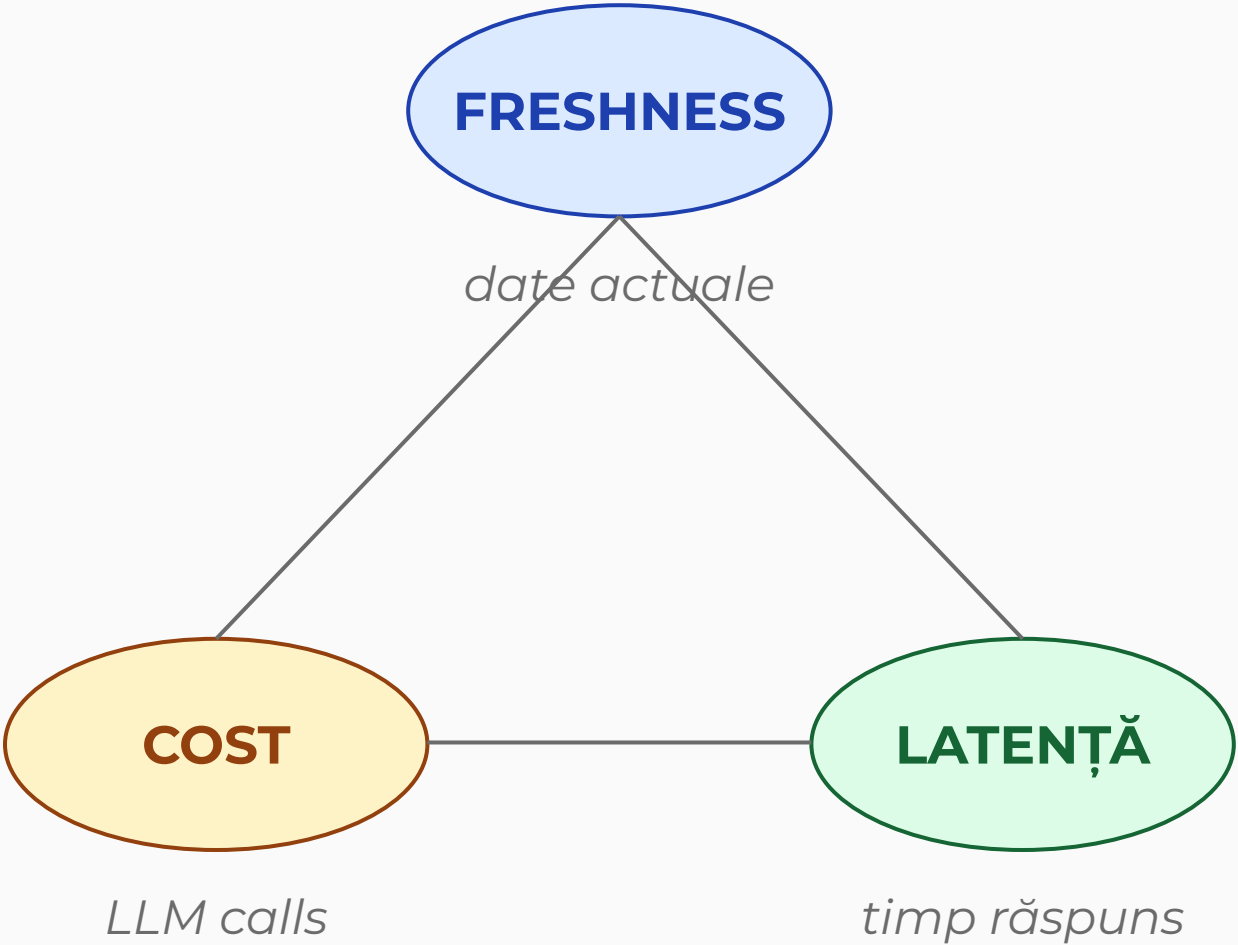
R Resource-intensive?

Costă mult timp / bani?

DA → **cachează** · NU → **recalculează**

Trade-offs

Triada cache-ului — alege 2 din 3, nu poți avea toate:



Strategie	Cost	Latență	Freshness
No cache	↑ Mare	↑ Mare	100% fresh
Agresiv	↓ Mic	↓ Mică	Stale risk
Cache + TTL	≈ Mediu	↓ Mică	Controlat ✓

🔑 Întrebarea cheie:

„Cât de stale este acceptabil?”

FAQ „Ce e VAT?” → stale 30 zile? **OK**
Curs valutar → stale 1 oră? **MAYBE**
Preț stoc → stale 1 minut? **RISKY**



Best of all worlds: Cache + TTL adaptat domeniului — controlezi stale vs cost vs latență.

Cache Invalidation

„There are only two hard things in CS: **cache invalidation** and naming things.”

— Phil Karlton

```
1
2 class SmartCache:
3     def __init__(self, session_factory: sessionmaker, threshold: float = 0.92, ttl: int = 3600):
4         self.session_factory = session_factory
5         self.threshold = threshold
6         self.ttl = ttl # time-to-live in secunde (1h)
7
8     def get(self, query: str) -> str | None:
9         emb = embed(query)[0]
10        cutoff = datetime.now(timezone.utc) - timedelta(seconds=self.ttl)
11        with self.session_factory() as db:
12            distance = SemanticCache.query_embedding.cosine_distance(emb)
13            row = db.execute(
14                select(SemanticCache, distance.label("dist"))
15                .where(SemanticCache.created_at >= cutoff) # ⚡ doar entries ne-expirate
16                .order_by(distance)
17                .limit(1)
18            ).first()
19
20        if row is None:
21            return None
22        cache, dist = row
23        if (1 - dist) < self.threshold:
24            return None
25        cache.hit_count += 1
26        db.commit()
27        return cache.response
28
29 # 🗑 Topic-based: șterge tot ce ține de un subiect (ex: doc sursă schimbat)
30 def invalidate_by_topic(self, topic: str) -> int:
31     with self.session_factory() as db:
32         result = db.execute(
33             delete(SemanticCache).where(SemanticCache.query_text.ilike(f"%{topic}%"))
34         )
35         db.commit()
36         return result.rowcount
37
38 # 🖱 Manual: user cere explicit "refresh" -> golește tot
39 def clear(self) -> None:
40     with self.session_factory() as db:
41         db.query(SemanticCache).delete()
42         db.commit()
43
44 # 🧹 Curățenie: șterge proactiv entries expirate (rulat periodic / cron)
45 def purge_expired(self) -> int:
46     cutoff = datetime.now(timezone.utc) - timedelta(seconds=self.ttl)
47     with self.session_factory() as db:
48         result = db.execute(delete(SemanticCache).where(SemanticCache.created_at < cutoff))
49         db.commit()
50         return result.rowcount
51
```



TTL

Time-to-live

Auto-expirare după N secunde

ttl=3600 (1 oră)



Topic-based

invalidate_by_topic()

Șterge entries legate de un subiect

DELETE WHERE query ILIKE %topic%



Manual

User-triggered

User cere explicit refresh

cache.clear() / "refresh"

Monitorizare: cache hit rate

„Dacă nu măsoři, nu știi dacă funcționează” — track hits, misses și savings:

```
1 @dataclass
2 class CacheMetrics:
3     hits: int = 0
4     misses: int = 0
5     cost_per_call: float = 0.01 # cost mediu estimat per apel LLM evitat ($)
6
7     def record_hit(self):
8         self.hits += 1
9
10    def record_miss(self):
11        self.misses += 1
12
13    @property
14    def total(self) -> int:
15        return self.hits + self.misses
16
17    @property
18    def hit_rate(self) -> float:
19        return self.hits / self.total if self.total else 0.0
20
21    @property
22    def estimated_savings(self) -> float:
23        # fiecare HIT = un apel LLM evitat = bani economisiți
24        return self.hits * self.cost_per_call
25
26    def report(self) -> dict:
27        return {
28            "hit_rate": f"{self.hit_rate * 100:.1f}%",
29            "hits": self.hits,
30            "misses": self.misses,
31            "estimated_savings": f"${self.estimated_savings:.2f}",
32        }
33
34
```

🎯 Target hit rates

Hit rate	Interpretare
< 20%	Ineficient
20–40%	Normal
40–60%	Bun (FAQ)
> 60%	Excelent ⭐

💡 **Sub 20%?** ajustează threshold sau audit query patterns

Best Practices finale

Reguli operaționale și stack recomandat pentru Document Analyst:

✓ DO

- Începe FĂRĂ cache, adaugă la pattern-uri repetitive
- Monitorizează hit rate și ajustează threshold
- Setează TTL rezonabil (1h – 24h)
- Loghează hits/misses pentru debugging
- Invalidează proactiv când datele sursă se schimbă

✗ DON'T

- Cache la tot (overhead > beneficiu pe query unice)
- Threshold prea mic → răspunsuri greșite
- TTL infinit → stale data
- Ignora monitorizarea (nu știi dacă funcționează)



Stack recomandat pentru Document Analyst

PostgreSQL

- **semantic_cache** (query → response)
- **embedding_cache** (text_hash → vector)
- **chat_messages** (conversation memory)

Parametri:

- Semantic threshold: **0.92**
- TTL: **3.600s (1 oră)**
- Max cache size: **10.000 entries**